

Machine Learning Assignment

Polynomial Regression

GitHub: https://github.com/praneethem/ML_Assignment

- Praneeth Marudi
- IMT2023555

1 Introduction

This assignment has two problems, and in both the objective is to predict a continuous target variable y using polynomial regression.

- **VAR1 (steam turbine):** six inputs $x_1, \dots, x_6$, and the target is the Net Power Score.
- **VAR2 (thermal reservoir):** three spatial inputs x_1, x_2, x_3 , and the target is the Thermal Anomaly Score.

Each problem has 1000 training rows with y and 1000 test rows without y . I worked in two steps.

Step 1: ordinary polynomial regression. Initial baselines were established using ordinary least squares (OLS) polynomial regression and searched over the degree and over feature subsets. Cross-validation picked **degree 4** for VAR1 (209 terms) and **degree 8** for VAR2 (164 terms). This was already a big improvement over a straight line, but it also showed a problem: at higher degrees the plain model overfits badly.

Step 2: regularised polynomial regression. To fix this, I kept the polynomial model but added a penalty on the size of the coefficients (Ridge, Lasso, Elastic Net, and a Lasso-then-Ridge combination called Relaxed Lasso). This let me use higher degrees safely. My final models are:

- **VAR1:** degree 5, **Relaxed Lasso** (Lasso $\alpha = 0.02$, then Ridge refit $\alpha = 0.1$). Lasso keeps 91 of the 461 terms.
- **VAR2:** degree 11, **Ridge** with $\alpha = 1.23285$ (363 terms).

Table 1 shows how the cross-validated error changed at each stage. All choices were made with cross-validation on the training data. The test labels are hidden and were never used.

Table 1: 5-fold cross-validation MSE at each stage (lower is better).

	Straight line	Best plain polynomial (Step 1)	Final regularised model (Step 2)
VAR1	8.847	0.835 (degree 4)	0.313 (62% lower than Step 1)
VAR2	31.298	0.295 (degree 8)	0.252 (15% lower than Step 1)

2 Exploratory Data Analysis (EDA)

I started by checking the files. None of the four files has missing values, all columns are numeric, and every feature lies between -1 and 1 . The training sets have no duplicate rows. The test sets contain 6 duplicate rows (VAR1) and 7 duplicate rows (VAR2).

The target in VAR1 has mean 0.81 and standard deviation 3.21 (range -9.92 to 12.07). In VAR2 it has mean 2.16 and standard deviation 6.63 (range -29.81 to 40.03).

Correlations. In VAR1 no single feature has a strong straight-line relation with y (the largest correlation is -0.32 , for x_5), while x_1 to x_4 are moderately correlated with each other (0.40 to 0.49). In VAR2, x_2 has the strongest link with y (0.51), and x_1 and x_3 are weak (Figure 1).

Shape of the relationship. The scatter plots (Figure 2) are clearly curved, and in VAR2 the spread of y grows towards the edges of the cube. A plain straight-line fit confirms this: its cross-validated R^2 is only 0.136 (VAR1) and 0.284 (VAR2). Going to degree 2 raises these to 0.692 and 0.547 , so a higher degree is needed.

Train versus test inputs. For VAR2 the test inputs look like the training inputs. For VAR1 they do not: the standard deviation of each feature is about 0.81 to 0.84 in the test set but about 0.71 to 0.72 in training, and 51% of the VAR1 test feature values sit exactly on ± 1 compared with 30% in training. So the VAR1 test

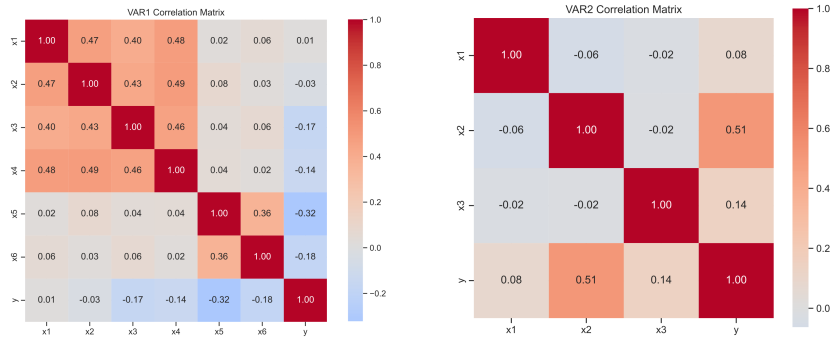


Figure 1: Correlation matrices for VAR1 (left) and VAR2 (right).

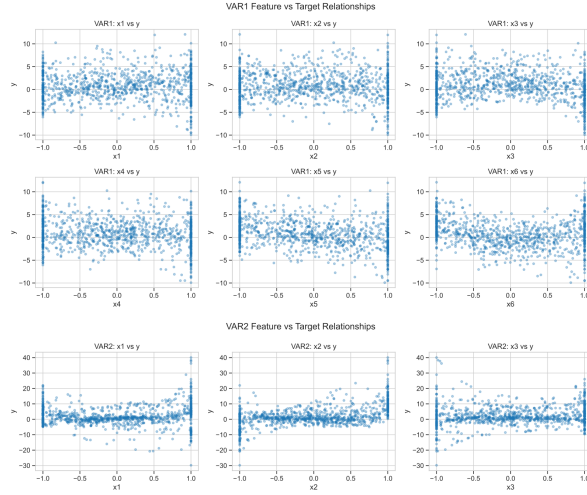


Figure 2: Each feature against y in the training data: VAR1 (top) and VAR2 (bottom).

points lie closer to the edges of the input range, where the training data is thinner. I come back to this in Section 6.

3 Step 1: Plain Polynomial Regression

Model. For a chosen degree d , `PolynomialFeatures` builds every term whose powers add up to at most d , including mixed terms such as $x_1x_2^2$. With k features the number of terms is

$$p(k, d) = \binom{k+d}{d} - 1,$$

for example $p(6, 4) = 209$, $p(6, 5) = 461$, $p(6, 6) = 923$, $p(3, 8) = 164$ and $p(3, 11) = 363$. In this step the terms are fitted with ordinary least squares (`LinearRegression`).

Tools. I standardised the polynomial terms (`StandardScaler`) because high-degree terms have very different sizes. I compared models with **5-fold cross-validation** (shuffled, `random_state=42`): the 1000 rows are split into 5 folds, so each round trains on 800 rows and validates on 200. Comparing training error with validation error shows underfitting (both high) and overfitting (training low, validation high). Scaling sits inside a `Pipeline`, so it is learned only from the training part of each fold.

Search. I tried every degree from 1 to 10 for VAR1 and 1 to 20 for VAR2 (the maximum degrees in the problem statements) using all features. Then I tried every non-empty subset of features with every degree: $63 \times 10 = 630$ configurations for VAR1 and $7 \times 20 = 140$ for VAR2. I picked the lowest mean validation MSE and also looked at neighbouring degrees to check that the choice is not a lucky spike.

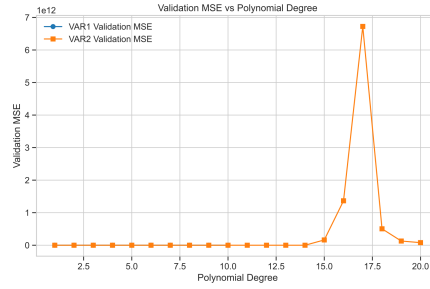


Figure 3: Plain polynomial regression: validation MSE against degree for VAR1 and VAR2 (linear scale). The VAR2 error at the highest degrees is so large that the other values look flat, so Table 2 gives the exact numbers for the useful range.

Table 2: Plain polynomial regression, 5-fold cross-validation with all features (VAR2 degrees 12 to 20 left out; their validation error is much larger and unstable).

degree	VAR1 (6 features)			VAR2 (3 features)		
	train MSE	val MSE	val R^2	train MSE	val MSE	val R^2
1	8.664	8.847	0.136	30.827	31.298	0.284
2	2.886	3.119	0.692	18.770	19.692	0.547
3	0.767	0.959	0.905	9.982	11.008	0.748
4	0.358	0.835	0.919	3.343	3.981	0.908
5	0.113	1.651	0.838	1.247	1.662	0.962
6	≈ 0	65.360	-5.348	0.454	0.647	0.985
7	≈ 0	6.989	0.320	0.230	0.338	0.992
8	≈ 0	4.312	0.585	0.164	0.295	0.993
9	≈ 0	3.923	0.618	0.144	0.306	0.993
10	≈ 0	4.347	0.582	0.125	0.383	0.991
11	—	—	—	0.107	0.525	0.988

VAR1. Validation error keeps falling until degree 4 (MSE 0.835, $R^2 = 0.919$) and then goes up. At degree 6 the model has 923 terms, which is more than the 800 training rows in each fold, so it memorises the training data (training MSE is zero) and the validation MSE jumps to 65.4. Degrees 7 to 10 stay in this memorising regime. Degree 3 is close behind (0.959), but degree 4 is better.

VAR2. Validation error drops smoothly to its minimum at degree 8 (MSE 0.295, $R^2 = 0.993$). Degrees 7 and 9 are close (0.338 and 0.306), so the choice is stable. After degree 9 the error rises again, and at the highest degrees the models become numerically unstable and perform far worse than degree 8.

Feature subsets. The best configurations used the full feature set. For VAR1 the best model without one feature (dropping x_4 , degree 3) has a validation MSE of 1.798, more than twice that of the full model. For VAR2 the ten best configurations all use all three coordinates. So I kept all features.

Where Step 1 ended. The best plain models were VAR1 degree 4 (MSE 0.835) and VAR2 degree 8 (MSE 0.295). But the degree acted like a hard switch. VAR1 went from 0.835 at degree 4 to 1.651 at degree 5 and 65.4 at degree 6, so I could not use the extra flexibility of higher degrees. This is where the regularised models come in.

4 Step 2: Regularised Polynomial Regression

Idea. A regularised model is still a polynomial regression, but when fitting it adds a penalty on the size of the coefficients. This stops the model from chasing noise, so a higher degree can help instead of hurt. The strength of the penalty is called α and is chosen by cross-validation. Because the penalty treats all coefficients alike, the standardisation step is now essential and not just a numerical convenience.

Models tried. All of them use the same pipeline as before (polynomial terms, then standardisation, then the model) and the same 5 folds (seed 42):

- **Ridge** (L2 penalty): shrinks all coefficients but keeps every term.
- **Lasso** (L1 penalty): shrinks coefficients and sets many exactly to zero, so it selects terms.
- **Elastic Net**: a mix of the two, controlled by an L1 ratio.

- **Relaxed Lasso** (VAR1): Lasso first picks which terms to keep, then a light Ridge model is refitted using only those terms. The refit shrinks the kept terms less than Lasso alone does.

Search. For Ridge and Lasso I used 11 values of α from 0.0001 to 10, with degrees 3 to 6 for VAR1 and 6 to 11 for VAR2. For Elastic Net I used α from 0.001 to 0.1 and L1 ratios 0.3, 0.5, 0.7 (degrees 4 to 6 for VAR1 and 8 to 11 for VAR2), plus a finer VAR1 search around the best point. For Relaxed Lasso I used degree 5 (the best degree for Lasso and Elastic Net on VAR1), six Lasso α values and five Ridge α values. In total this was about 430 regularised configurations. Finally, for the VAR2 Ridge model I tried a few α values between the grid points around the best one ($\alpha = 3.0$ on the grid) and kept $\alpha = 1.23285$.

Table 3: Effect of degree under regularisation. Ridge and Lasso are each shown at their best α for that degree. Validation MSE, 5-fold cross-validation.

VAR1				VAR2			
degree	plain	Lasso	Ridge	degree	plain	Ridge	Lasso
3	0.959	0.954	0.959	6	0.647	0.646	0.647
4	0.835	0.629	0.746	7	0.338	0.338	0.340
5	1.651	0.322	0.525	8	0.295	0.280	0.280
6	65.360	0.342	0.595	9	0.306	0.258	0.259
				10	0.383	0.261	0.262
				11	0.525	0.252	0.258

Table 3 shows the main effect. Without a penalty, VAR1 degree 6 gives 65.4. With Lasso it gives 0.342, and degree 5 gives 0.322. For VAR2 the penalty removes the rise after degree 8, and the error keeps improving slowly up to degree 11.

Table 4: Best result of each method (5-fold cross-validation, all features).

Problem	Method	Settings	val MSE	val R^2
VAR1	Plain polynomial	degree 4	0.8351	0.9186
VAR1	Ridge	degree 5, $\alpha = 10$	0.5253	0.9484
VAR1	Elastic Net	degree 5, $\alpha = 0.01$, L1 ratio 0.7	0.3263	0.9680
VAR1	Lasso	degree 5, $\alpha = 0.01$	0.3218	0.9684
VAR1	Relaxed Lasso	degree 5, Lasso $\alpha = 0.02$, Ridge $\alpha = 0.1$	0.3134	0.9693
VAR2	Plain polynomial	degree 8	0.2948	0.9932
VAR2	Lasso	degree 11, $\alpha = 0.001$	0.2575	0.9940
VAR2	Ridge	degree 11, $\alpha = 1.23285$	0.2517	0.9942
VAR2	Elastic Net	degree 11, $\alpha = 0.001$, L1 ratio 0.3	0.2512	0.9942

VAR1. Dropping terms clearly helps here. Ridge keeps all 461 terms and only reaches 0.525, while Lasso and Elastic Net, which drop terms, reach about 0.32. The finer Elastic Net search (L1 ratio up to 0.9) gave 0.3227, still behind Relaxed Lasso. Relaxed Lasso was best at 0.3134: Lasso keeps 91 of the 461 terms (1 of degree 1, 4 of degree 2, 5 of degree 3, 32 of degree 4 and 49 of degree 5, involving all six inputs), and the Ridge refit then corrects the shrinkage on those terms. That is 62% lower than the best plain model.

VAR2. Here the methods are very close (0.2512 to 0.2575), so dropping terms does not give an obvious gain over Ridge. Elastic Net has the lowest VAR2 cross-validation MSE (0.2512), while Ridge is very close (0.2517) and was chosen for simplicity, because it has only one setting to tune. The Ridge error is flat around its best point: $\alpha = 1, 1.23$ and 3 give 0.2525, 0.2517 and 0.2524. A difference of 0.0005 (0.2%) is not a reliable reason to prefer one model. Compared with the best plain model (0.2948) the final error is about 15% lower.

5 Final Models and Checks

The final models are VAR1 Relaxed Lasso (degree 5, 91 of 461 terms) and VAR2 Ridge (degree 11, $\alpha = 1.23285$, 363 terms). Although Elastic Net had a marginally lower cross-validation MSE for VAR2, I chose Ridge, degree 11, $\alpha = 1.23285$, for simplicity. I refitted these pipelines on all 1000 training rows and used them to predict the 1000 test rows of each problem. The cross-validation scores in Table 4 (bold rows) are my estimate of out-of-sample performance. The true test performance is not known because the test labels are hidden. The numbers in Table 5 are measured on the training data itself, so they are naturally better (training MSE 0.228 against a cross-validation MSE of 0.313 for VAR1, and 0.165 against 0.252 for VAR2).

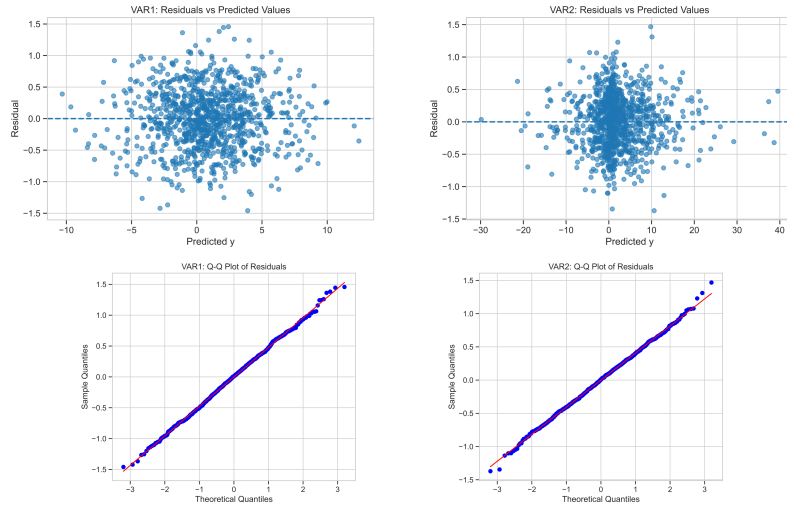


Figure 4: Final models on the training data. Top: residuals against predicted values. Bottom: Q-Q plots of the residuals. VAR1 on the left, VAR2 on the right.

Table 5: Fit and residuals of the final models on the training data.

	MSE	RMSE	R^2	mean res.	median res.	min res.	max res.
VAR1 (Relaxed Lasso)	0.228	0.477	0.978	0.000	0.010	-1.460	1.457
VAR2 (Ridge)	0.165	0.406	0.996	0.000	0.008	-1.371	1.468

The residuals (Figure 4, top) are centred around zero and show no visible curvature or funnel shape, so the models are not missing any obvious structure. The Q-Q plots (bottom) are reasonably close to a straight line, suggesting that the residuals are approximately normally distributed.

6 Limitations

- **VAR1 test distribution shift.** The VAR1 test inputs have a wider spread and lie closer to the edges of the input range than the training inputs. Therefore, the cross-validation score may be somewhat optimistic for the hidden test set. Eleven of the 1000 VAR1 predictions fall outside the training target range $(-9.92, 12.07)$. For VAR2, only one prediction falls outside the training range, and only slightly $(-29.85$ versus $-29.81)$.
- **Slightly optimistic model-selection scores.** About 1200 configurations were compared using the same five CV folds, so the best CV scores may be somewhat optimistic. The VAR1 improvement from 0.835 to 0.313 is large relative to this effect, while the VAR2 improvement is smaller and should therefore be interpreted as a modest gain.
- **Dependence on one CV shuffle.** All results use the same shuffled 5-fold split with `random_state=42`. Different fold assignments could produce slightly different scores. The VAR2 degree search was also stopped at degree 11 for the regularised models.

7 Code and Files

All the code is in the notebook `ML_Assignment.ipynb` (see the GitHub link at the top). It reads the four personalised training and test datasets from the `Dataset/` directory and writes the prediction files `IMT2023555_pred_var1.csv` and `IMT2023555_pred_var2.csv`. Each prediction file has 1000 rows in the same order as the test file, with one column `y` and no missing values, and the notebook checks this automatically. As a last check, I refitted both final models separately from the training files and got the same predictions as in the submitted files.